Progress Report #1

Alexandro Segura
School of Engineering and Computer Science
Alexandrosegura@oakland.edu

1. Project Overview

This project looks at whether network scanning can be detected in TCP traffic using software like Wireshark and Nmap. I believe that scanning is a common and powerful way to find open ports, but attackers also use it to gather information. I am comparing normal TCP connections with Nmap TCP connect scans and SYN scans to find which packet patterns make a scan easier to recognize in order to predict or identify if your network is at risk.

2. Progress and Related Work

So far I have chosen the topic, written the project proposal, and worked on the project website. I also set up a small VirtualBox lab with Kali Linux as the scanning machine and Ubuntu as the target. All tests will be run on a private virtual network so the scanning stays inside my control. Nmap's documentation describes the difference between a TCP connect scan, which completes the TCP handshake, and a SYN scan, which usually stops before the connection is complete. I've read about a similar situation, Jung et al. (2004) studied scan detection based on connection behavior. I've decided to limit the project to showing patterns in Wireshark, and not go into the world of intrusion detection.

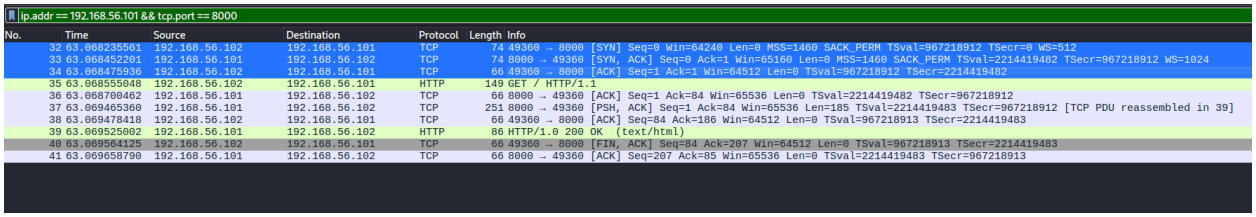
3. Testing

The test used Kali which in this project will be (192.168.56.102), and will act as the scanner and Ubuntu (192.168.56.101) as the target. A python test web server was also made on the Ubuntu system on TCP port 8000. During the test I captured three types of traffic in Wireshark, a normal HTTP request, an Nmap TCP connect scan (-sT), and an Nmap SYN scan (-sS). These scans were done through ports 7995 to 8005.

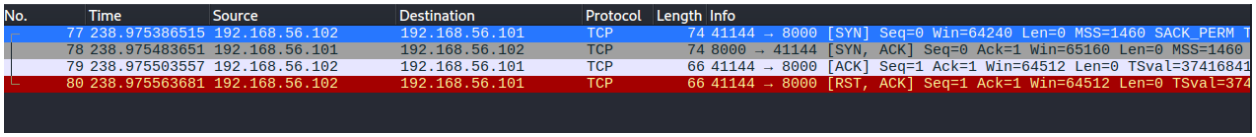
4. Challenges and Next Steps

These first tests show that the two scan types produce different packet patterns. I have not yet explored detection accuracy or false positives, though I believe I will with time. An important aspect of these tests is that normal applications can also open multiple connections, so filtering based only on SYN packets could flag regular traffic. I plan to compare the captures more closely and write a simple detection rule/filter to test it against both normal and scanning traffic.

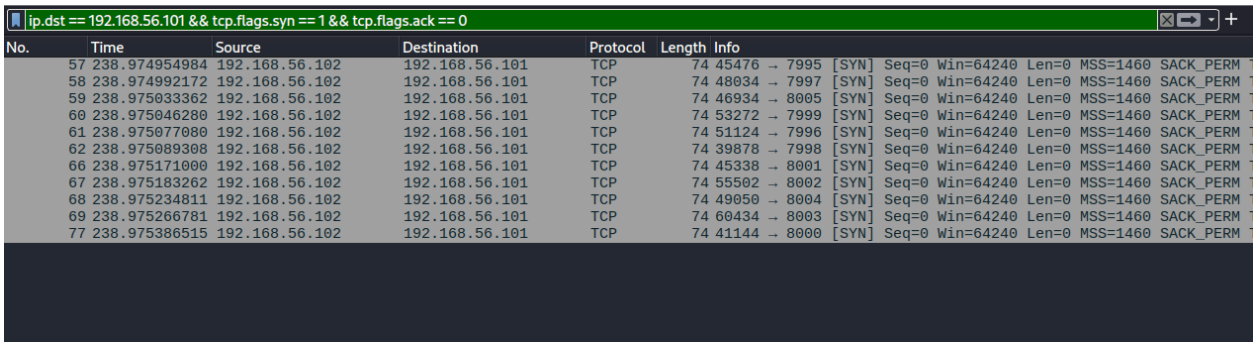
Wireshark Evidence



1. Normal HTTP connection: SYN, SYN/ACK, ACK, and HTTP 200 OK.



2. TCP connect scan: completed handshake followed by RST/ACK.



3. TCP connect scan: SYN packets sent across ports 7995–8005.

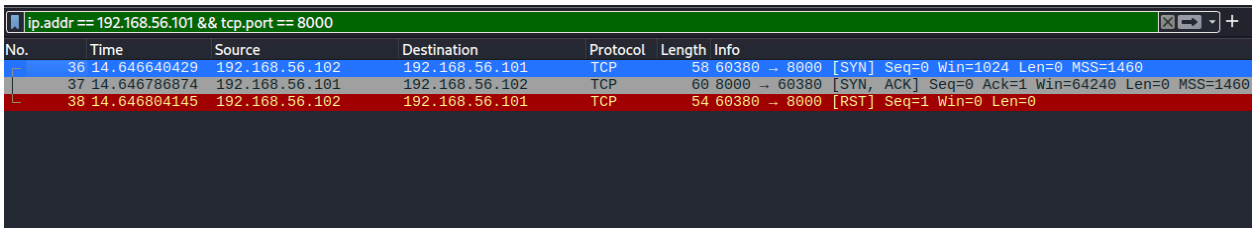


Figure 4. SYN scan: SYN, SYN/ACK, and RST without a completed handshake.

References

Nmap. Port Scanning Techniques. https://nmap.org/book/man-port-scanning-techniques.html

Wireshark. TCP Display Filter Reference. https://www.wireshark.org/docs/dfref/t/tcp.html